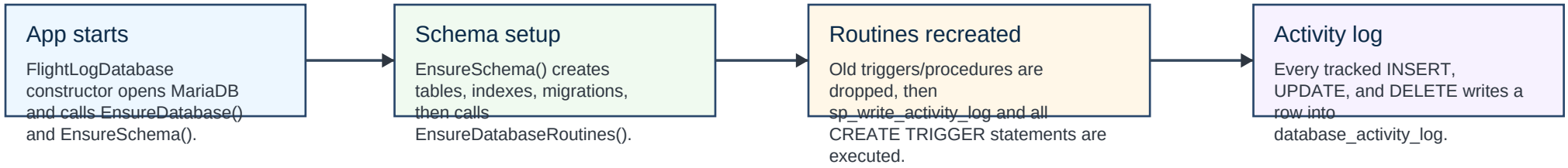
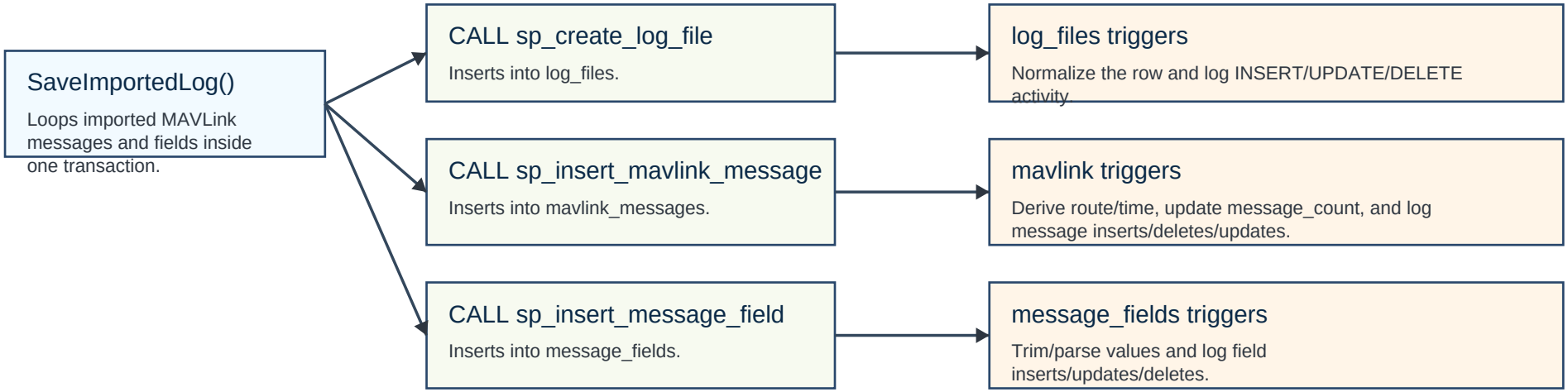
OpenHD FlightLog SQL Trigger Logging

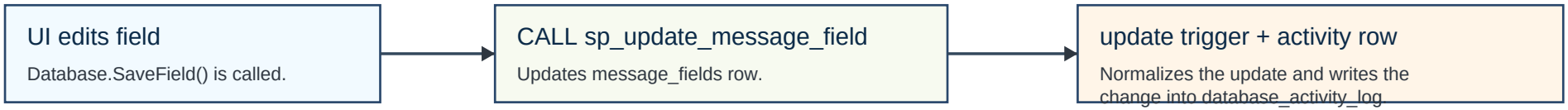
How triggers and stored procedures track database activity automatically



Import path: C# calls stored procedures; trigger execution is automatic.



Edit path: field edits also pass through database rules.



Implementation Snippets

Source: OpenHdFlightLog/Services/FlightLogDatabase.cs

1. Activity log table and writer procedure

```
CREATE TABLE IF NOT EXISTS database_activity_log (
    id BIGINT PRIMARY KEY AUTO_INCREMENT,
    changed_at DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
    table_name VARCHAR(64) NOT NULL,
    activity_type VARCHAR(16) NOT NULL,
    row_id BIGINT NULL,
    summary TEXT NOT NULL
);

CREATE PROCEDURE sp_write_activity_log(...)
BEGIN
```

2. Routines are recreated during schema setup

```
private void EnsureDatabaseRoutines(MySqlConnection connection)
{
    string[] routineDrops =
    [
        "DROP TRIGGER IF EXISTS trg_log_files_before_insert;",
        "DROP TRIGGER IF EXISTS trg_log_files_after_insert;",
        "DROP PROCEDURE IF EXISTS sp_write_activity_log;"
    ];

    foreach (var sql in routineDrops)
        ExecuteNonQuery(connection, sql, sql.TrimEnd(';'));
```

3. C# calls procedures, triggers fire automatically

```
command.CommandText =
    "CALL sp_insert_mavlink_message(...);";

-- inside the procedure
INSERT INTO mavlink_messages (...)
VALUES (...);

-- then MySQL runs the trigger
CREATE TRIGGER trg_mavlink_messages_after_insert
AFTER INSERT ON mavlink_messages
```

4. Edit tracking trigger example

```
CREATE TRIGGER trg_user_variables_after_update
AFTER UPDATE ON user_variables
FOR EACH ROW
BEGIN
    CALL sp_write_activity_log(
        'user_variables',
        'UPDATE',
        NEW.id,
        CONCAT('Variable geaendert: ',
               OLD.name, ' -> ', NEW.name));
```

Why we need them

- Fulfills the assignment requirement for a persistent activity log table.
- Tracks inserts, updates, and deletes automatically in MariaDB, not manually in UI code.
- Uses stored procedures for write paths and a stored procedure for trigger logging.
- Shows the protocol in the DB Activity tab: table, operation, row id, time, summary.
- Keeps the older normalization triggers for route, timestamps, counts, and field cleanup.

Old vs New Database Activity Tracking

What changed compared to a non-trigger or UI-only logging approach

Older / non-trigger approach

Application code writes data and optionally writes debug text.
Tracking depends on every C# write path remembering to log manually.

New trigger-based approach

Stored procedures perform writes; triggers automatically log the result.
Tracking lives in MariaDB and is persisted in database_activity_log.

Before: manual or missing tracking in C#

```
public void DeleteVariable(long variableId)
{
    command.CommandText =
        "DELETE FROM user_variables WHERE id = @id;";
    command.ExecuteNonQuery();

    // Only UI/debug output. If another code path deletes
    // data and forgets this line, no persistent audit row exists.
    Log("SQL WRITE", $"DELETE user_variables id={variableId}");
}
```

After: database-level tracking

```
CREATE TRIGGER trg_user_variables_after_delete
AFTER DELETE ON user_variables
FOR EACH ROW
BEGIN
    CALL sp_write_activity_log(
        'user_variables',
        'DELETE',
        OLD.id,
        CONCAT('Variable geloescht: ', OLD.name));
END
```

Practical differences

Without trigger logging

- Logs are application behavior, not database behavior.
- Direct SQL changes are invisible unless the caller logs manually.
- A missed Log(...) call means the change is not traceable.
- DebugEvents disappear when the app restarts.
- Does not clearly satisfy the assignment requirement for a log table.

With trigger logging

- Logs are enforced by MariaDB whenever tracked rows change.
- Insert, update, and delete activity is written automatically.
- database_activity_log remains available after restart.
- The DB Activity tab can show an audit trail from the table.
- Matches: Log table + triggers + stored procedures.